LLM Inference Request Scheduling under GPU Memory Limits

Abhishri Krishna (24CS2004) | Vedika Choudhary (24CS3060) | Sonali Swaraj (24CS3052)
Manya Dixit (24CS2026) | Yasashwitha (24CS2015)
Department of Computer Science and Engineering, RGIPT
Presented to Dr. Aakash Yadav

Abstract

We implement and evaluate list-scheduling policies for offline LLM request scheduling under a discrete GPU-memory budget. We compare FCFS, SDF, MAGLS, event-jump E-MAGLS, and a ranking portfolio with local search. Across 40 synthetic offline workloads (8-64 requests, 10 seeds per size), every schedule was feasible. At 64 requests, MAGLS reduced mean total flow by 20.4% vs. FCFS and 8.0% vs. SDF; the portfolio reduced it by 20.6% and 8.3%. On 10 six-request cases, the heuristics' mean gaps to proven MILP optima were 15.90% (FCFS), 7.94% (SDF), 2.60% (MAGLS), and 1.36% (portfolio). For five cases each at 30, 40, and 50 requests, the MILP proved 5, 2, and 3 optima within 15 seconds; the largest residual solver gap was 0.70%. E-MAGLS matched MAGLS schedules but ran 1.74 times slower at n=64. Results use synthetic inputs and a scheduling abstraction, not GPU-engine measurements.

Keywords: LLM serving; KV cache; total flow time; memory-aware scheduling; synthetic evaluation

1. Introduction

Autoregressive inference retains a key-value (KV) cache for each active request. Prompt tokens determine the initial cache footprint, and decoding adds cache state over time. Continuous batching and paged cache management improve utilization, but admission and scheduling still interact with memory capacity. Orca schedules at iteration granularity, while vLLM's PagedAttention manages KV-cache blocks and reduces fragmentation (Yu et al., 2022; Kwon et al., 2023). More recent systems optimize throughput, latency, or goodput through chunked prefill, phase disaggregation, and scheduling policies (Agrawal et al., 2024; Zhong et al., 2024).

This project isolates a smaller combinatorial question: given request sizes, arrivals, and decode lengths, when should each non-preemptive request start to minimize summed completion-minus-arrival time while respecting memory and a batch cap? The model is useful for algorithm comparison, but its unit-time and linear-cache assumptions are not a latency model. We therefore report synthetic scheduling metrics and do not interpret them as measured GPU performance.

The work product is an independently checked simulator, four ranking policies plus a portfolio/local-search procedure, and a seeded experiment harness. We report results against FCFS and SDF, including a finite-batch arrival check. We also give a hardness reduction for decode durations greater than one, while separating that inherited single-machine hardness from hardness caused specifically by memory growth.

2. Problem Model and Related Work

2.1 Discrete-time formulation

Let I be the requests. Request i has prompt length p_i, decode duration d_i, and arrival iteration a_i. A schedule chooses an integer start s_i >= a_i. It is active at iterations s_i,...,s_i+d_i-1, completes at C_i=s_i+d_i-1, and contributes flow F_i=C_i-a_i. At active time t, its memory is p_i+(t-s_i). For GPU memory capacity M and optional batch limit B, feasibility requires, for every t, sum of active memory <= M and the number of active requests <= B. The objective is minimize sum_i F_i. Memory is measured in abstract token-equivalent units; page/block rounding and fragmentation are excluded.

The team convention starts a request at its arrival iteration (a_i), consistent with the repository simulator and hand example. Each request runs without preemption on one abstract GPU; arrival times, prompt/decode lengths, M, and B are known; every request must fit alone. Iterations have equal duration. These assumptions make the model reproducible, but omit variable iteration latency, preemption, recomputation, chunked prefill, uncertain output lengths, and multiple GPUs.

2.2 Position relative to prior work

The model is an abstraction of memory-constrained admission and decode scheduling, not a replacement for production serving. PagedAttention uses block-based allocation; Orca uses iteration-level scheduling; Sarathi-Serve balances throughput and latency with chunked-prefill scheduling; DistServe separates prefill and decode to target service-level objectives (Kwon et al., 2023; Yu et al., 2022; Agrawal et al., 2024a; Zhong et al., 2024). FastServe explores preemption and token-level scheduling (Wu et al., 2026). These systems model execution and service objectives beyond our single-resource discrete abstraction. The supplied project base paper studies variable prefill and decode lengths (LLM Serving Optimization with Variable Prefill and Decode Lengths, 2025); an exact claim-by-claim comparison and implementation of its algorithms are not included in this implementation.

Other recent systems address dynamic rescheduling and fragmentation, prompt sharing, phase splitting, scalable simulation, structured generation, or adapter multiplexing (Sun et al., 2024; Srivatsa et al., 2024; Patel et al., 2024; Agrawal et al., 2024b; Zheng et al., 2024; Chen et al., 2024; Qin et al., 2025). These systems measure inference behavior on serving engines or traces; our synthetic evaluation does not claim comparable latency or throughput. For foundations, single-machine scheduling surveys and complexity results provide context for release-date and completion-time objectives (Smith, 1956; Graham et al., 1979; Lenstra et al., 1977; Lawler et al., 1993; Goemans et al., 2002; Brucker, 2007). Dynamic storage allocation and bin packing study time-varying resource occupancy and fragmentation (Coffman et al., 1983; Garey & Johnson, 1979; Ivkovi■ & Lloyd, 1998; Buchsbaum et al., 2004; Murthy & Bhattacharyya, 1999; Mehta & DeWitt, 1993). They motivate our resource constraint but differ in objective and model details; this paper does not claim new results for those problems.

2.3 Hardness with decode duration greater than one

Theorem. The total-flow-time decision problem is strongly NP-hard even when p_i=1, d_i>=2 for every request, and B=1. Proof sketch. Start from the strongly NP-hard single-machine problem 1|r_j|sum C_j, which remains strongly NP-hard with unit weights (Lenstra et al., 1977; Goemans et al., 2002). Since sum r_j is constant, minimizing sum(C_j-r_j) is equivalent. For each job j with processing q_j and release r_j, create a request with p_i=1, d_i=2q_j, and a_i=2r_j; set B=1 and M=2 max_j q_j. Every request fits alone, and B=1 makes memory nonbinding. Doubling source times doubles total flow. Conversely, an optimal target schedule can be left-justified; starts are even releases or even completions, and halving gives a source schedule. A source threshold K on sum C_j maps to 2(K-sum_j r_j). This proves hardness in a growing-decode regime, but inherits release-date single-machine hardness rather than isolating the growing-memory constraint.

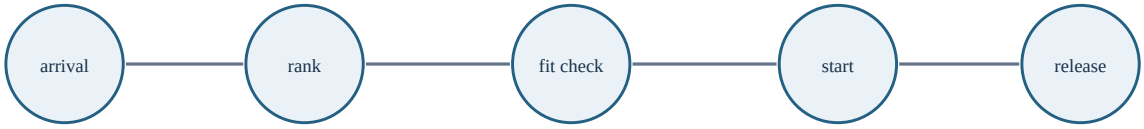
The d_i=1 reduction in the project draft separately gives the memory-packing hardness case. These reductions identify two distinct restrictions in the general problem; neither establishes that the time-varying memory constraint alone is strongly hard when B is unlimited and all decode lengths exceed one.

3. Methods

3.1 Algorithms and feasibility

All policies use the same deterministic priority order and earliest-feasible placement objective. FCFS orders by arrival then ID; SDF orders by decode duration, arrival, and ID; MAGLS (memory-aware greedy list scheduling) orders by memory-time area A_i=p_i d_i+d_i(d_i-1)/2, arrival, and ID. The area is the sum of a request's memory over its run. E-MAGLS (event-driven MAGLS) keeps this ranking but jumps over contiguous forbidden start intervals: at a violating iteration t, the memory or batch constraint identifies an interval of starts that cannot fit, and the search resumes after that interval. This preserves earliest-fit behavior. A parity check found identical MAGLS/E-MAGLS schedules on 100 seeded instances with and without arrivals and finite B.

The portfolio computes FCFS, SDF, MAGLS, and area-per-wait rankings, retains the lowest-flow feasible schedule, then tests adjacent pair swaps once and keeps the best strict improvement. Since every candidate placement checks the complete run window and the checker independently recomputes memory and activity, feasibility is enforced by construction and rechecked after scheduling. This is a feasibility property of the implementation, not an approximation ratio.



A start is accepted only if its full decode window fits memory M and batch B.

Figure 1. Scheduling pipeline and full-window feasibility check.

3.2 Experimental design

The standard-library Python harness uses fixed seeds and generates p_i from a clipped log-normal distribution and d_i from a second clipped log-normal distribution. Each of 10 seeds is run at n in {8,16,32,64}; M is set from the maximum individual footprint and 42% of aggregate prompt footprint; B is unlimited. This yields 40 offline workloads x five policies = 200 schedules. A second set contains 10 workloads of 16 requests, nonzero exponentially sampled arrivals, and B=4, adding 50 schedules. Every schedule is checked independently. Reported runtime is wall-clock Python runtime on the same host; no GPU or LLM is invoked. The online set is a small feasibility/behavior check, not a full online benchmark.

Metrics are total flow time, mean per-request flow, mean over-instance 95th-percentile flow, utilization defined as summed memory profile divided by M times the inclusive makespan horizon, and runtime. The sparse time-indexed MILP uses SciPy's HiGHS-backed solver. Start domains are safely bounded by a feasible MAGLS objective; every schedule is checked independently. Ten six-request cases are also solved by exhaustive branch-and-bound, and MILP/exhaustive optima agree exactly. A separate scaling run tests five seeded workloads each at n=30, 40, and 50 with a 15-second solver limit; unfinished runs report the solver's incumbent, lower bound, and gap rather than being labelled optimal.

4. Results

4.1 Offline workloads

Policy	Flow	vs FCFS	vs SDF	P95 flow	Utilization	Runtime (ms)
FCFS	334.5	+0.0%	-15.6%	10.7	64.9%	15.58
SDF	289.4	+13.5%	+0.0%	11.7	62.2%	14.98
MAGLS	266.3	+20.4%	+8.0%	11.4	62.6%	13.83
E-MAGLS	266.3	+20.4%	+8.0%	11.4	62.6%	23.88
Portfolio+LS	265.4	+20.7%	+8.3%	11.5	63.0%	1050.72

Table 1. Means over 10 offline instances at n=64. Percentage differences are computed from mean total flow; negative values indicate improvement over the named baseline.

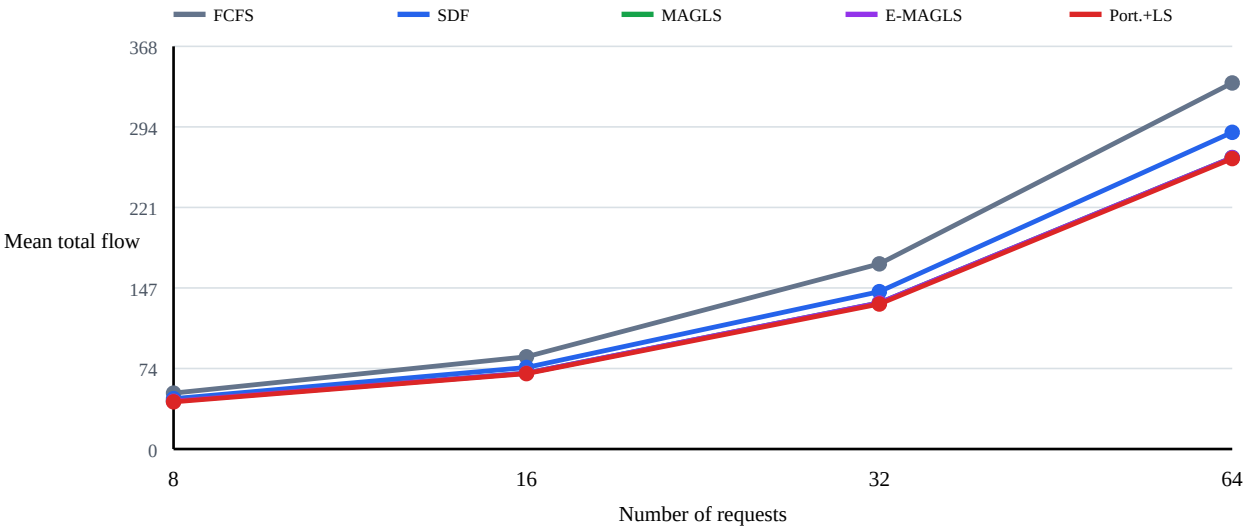


Figure 2. Mean total flow time on offline synthetic workloads (10 seeds per size). Lower is better.

At n=64, FCFS averages 334.5 flow units and SDF 289.4. MAGLS averages 266.3, a 20.4% reduction from FCFS and 8.0% from SDF. Portfolio plus local search averages 265.4, improving by 20.7% and 8.3%. All methods were feasible on all 40 offline instances. The portfolio gains only 0.9 additional flow units over MAGLS on average at n=64, while requiring 76 times as much runtime.

The tail result differs from the sum objective: the n=64 mean instance-level P95 is 10.7 for FCFS, 11.4 for MAGLS, and 11.5 for the portfolio. MAGLS also has lower memory utilization than FCFS (62.6% vs 64.9%), so the current objective does not reward high utilization directly. This indicates an objective tradeoff rather than universal dominance.

4.2 Exact tiny-instance comparison

Policy	Mean gap to optimum	Optimal schedules
FCFS	15.90%	0/10
SDF	7.94%	4/10
MAGLS	2.60%	6/10
Portfolio+LS	1.36%	8/10

Table 2. Heuristic gaps to the proven time-indexed MILP optimum; 10 synthetic instances with six requests each, with each optimum cross-checked by exhaustive branch-and-bound.

The MILP and exhaustive search agreed exactly on all 10 optima. MAGLS is within 2.60% of the optimum on average in this small set, while the portfolio is within 1.36% and matches the optimum on eight of 10 cases. The exhaustive check provides an independent validation of the time-indexed model for these instances; it does not scale to the larger workloads.

4.3 Nonzero arrivals and finite batch cap

Policy	Total flow	Mean flow	P95 flow	Utilization	Runtime (ms)
FCFS	64.9	4.06	9.8	39.0%	1.50
SDF	64.9	4.06	14.8	38.4%	1.50
MAGLS	65.8	4.11	14.6	38.0%	1.53
E-MAGLS	65.8	4.11	14.6	38.0%	1.77
Portfolio+LS	59.8	3.74	11.8	39.0%	24.36

Table 3. Means over 10 synthetic arrival workloads (n=16, B=4). Every returned schedule passed the checker.

In the finite-B set, the portfolio averages 59.8 total flow versus 64.9 for FCFS and 65.8 for MAGLS. Its average P95 is 11.8; FCFS is 9.8. Across all 50 returned schedules in this set, all were feasible. Since the workload sample is small and synthetic, differences are descriptive; no statistical significance claim is made.

4.4 ILP scaling

Requests	Proven / 5	Mean solve (s)	Max MIP gap	MAGLS gap*	Portfolio gap*
30	5/5	4.20	0.00%	1.74%	1.01%
40	2/5	13.21	0.70%	2.39%	2.12%
50	3/5	7.70	0.55%	1.60%	1.38%

Table 4. Five seeded offline workloads per size, 15-second solver limit. *Heuristic gaps use the proven optimum when available and the best MILP bound otherwise; for unfinished solves they are lower-bound gaps, not exact optimality gaps.

The MILP proved all n=30 cases, two of five at n=40, and three of five at n=50. The maximum reported MILP gap among unfinished instances was 0.70%. All 15 workloads returned feasible incumbents. Portfolio and MAGLS are close to solver bounds in these instances, but a small gap to a bound does not prove a similarly small gap to the unknown optimum when the MILP has not closed.

5. Discussion and Conclusions

5.1 Interpretation and limitations

Area ranking improves mean summed flow in the offline synthetic workloads, consistent with preferring requests with smaller cumulative memory footprint. This is not a proof that area is the right ranking for a general instance: the area ignores arrival pressure and can increase the waiting time of some requests. The n=64 P95 comparison confirms that lower aggregate flow does not imply better tail flow. The portfolio/local-search step finds modest additional mean gains but has a much higher runtime; its usefulness depends on whether that gain justifies the added compute.

The evaluation has important scope limits. First, the generated distribution is not a trace collected from vLLM, ShareGPT, or another serving system. Second, abstract memory units and unit-length iterations do not capture bytes per token, model-layer KV size, batch-dependent decode time, prefill cost, or cache-block fragmentation. Third, decode lengths are known, and jobs never preempt. Fourth, E-MAGLS uses direct scans rather than a segment tree and is 1.74 times slower than MAGLS at n=64; no speed-up is demonstrated. Fifth, only five workloads per larger size were tested; some MILPs timed out with small residual bounds, and exhaustive optimum checks cover six requests only. The algorithms from the supplied base paper were not implemented. The d_i>=2 hardness reduction uses B=1 and leaves memory nonbinding, so hardness specifically due to growing memory with an unlimited batch remains open.

A production-relevant follow-up should calibrate iteration time and cache demand from a real serving engine, compare the base-paper algorithms, include preemption or chunked prefill, vary decode-length prediction error, and evaluate serving traces with a finite batch cap. E-MAGLS would need a segment tree or another accelerated fit data structure before a speedup can be claimed. A theoretical extension should isolate hardness due specifically to growing memory with unlimited B and d_i>1; the present proof does not do so.

5.2 Conclusion

This project implemented a common feasibility-checked scheduler and compared FCFS, SDF, MAGLS, event-jump E-MAGLS, and a ranking portfolio with local search. On 40 offline synthetic instances, all schedules were feasible, and MAGLS reduced mean total flow by 20.4% versus FCFS at n=64, while the portfolio achieved a 20.6% reduction at substantially higher runtime. The portfolio was 1.36% above proven six-request MILP optima on average. At n=30, 40, and 50, the time-limited MILP proved 5/5, 2/5, and 3/5 instances. E-MAGLS preserved MAGLS schedules but did not improve runtime. The portfolio's P95 was worse than FCFS. Results support memory-time ranking on these generated workloads, not production latency or universal improvement. Evaluation on serving traces and comparison with the base-paper algorithms remain necessary.

Acknowledgments

We thank Dr. Aakash Yadav for feedback on the proposed project and manuscript. The authors are the student team listed on the title page.

References

Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., Tumanov, A., & Ramjee, R. (2024). Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation.

Garey, M. R., & Johnson, D. S. (1979). Computers and intractability: A guide to the theory of NP-completeness. W. H. Freeman.

Goemans, M. X., Queyranne, M., Schulz, A. S., Skutella, M., & Wang, Y. (2002). Single-machine scheduling with release dates. SIAM Journal on Discrete Mathematics, 15(2), 165-192. https://doi.org/10.1137/S089548019936223X

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. Proceedings of the 29th Symposium on Operating Systems Principles. https://doi.org/10.1145/3600006.3613165

Lenstra, J. K., Rinnooy Kan, A. H. G., & Brucker, P. (1977). Complexity of machine scheduling problems. Annals of Discrete Mathematics, 1, 343-362.

Lawler, E. L., Lenstra, J. K., Rinnooy Kan, A. H. G., & Shmoys, D. B. (1993). Sequencing and scheduling: Algorithms and complexity. In S. C. Graves, A. H. G. Rinnooy Kan, & P. H. Zipkin (Eds.), Handbooks in Operations Research and Management Science (Vol. 4, pp. 445-522). Elsevier.

Wu, B., Zhong, Y., Zhang, Z., Liu, S., Liu, F., Sun, Y., Huang, G., Liu, X., & Jin, X. (2023). Fast distributed inference serving for large language models. arXiv:2305.05920.

Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., & Ha, S. (2022). Orca: A distributed serving system for transformer-based generative models. Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation, 521-538.

Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., & Zhang, H. (2024). DistServe: Disaggregating prefill and decoding for goodput-optimized LLM serving. Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation, 193-210.

Wang, M., Ye, Y., & Zhou, Z. (2025). LLM Serving Optimization with Variable Prefill and Decode Lengths. arXiv:2508.06133. https://arxiv.org/abs/2508.06133

Baker, B. S., Brown, D. J., & Katseff, H. P. (1985). A 5/4 algorithm for two-dimensional packing. Journal of Algorithms, 6(4), 596-608.

Brucker, P. (2007). Scheduling algorithms (5th ed.). Springer.

Coffman, E. G., Garey, M. R., & Johnson, D. S. (1983). Approximation algorithms for bin packing: A survey. In D. S. Hochbaum (Ed.), Approximation Algorithms for NP-hard Problems. PWS.

Conway, R. W., Maxwell, W. L., & Miller, L. W. (1967). Theory of scheduling. Addison-Wesley.

Graham, R. L., Lawler, E. L., Lenstra, J. K., & Rinnooy Kan, A. H. G. (1979). Optimization and approximation in deterministic sequencing and scheduling: A survey. Annals of Discrete Mathematics, 5, 287-326.

Ivkovi■, Z., & Lloyd, E. L. (1998). Fully dynamic algorithms for bin packing: Being (mostly) myopic helps. SIAM Journal on Computing, 28(2), 574-611.

Mehta, A., & DeWitt, D. J. (1993). Dynamic memory allocation for multiple-query workloads. Proceedings of the 19th International Conference on Very Large Data Bases, 354-367.

Moore, J. M. (1968). An n job, one machine sequencing algorithm for minimizing the number of late jobs. Management Science, 15(1), 102-109.

Murthy, P. K., & Bhattacharyya, S. S. (1999). Approximation algorithms and heuristics for the dynamic storage allocation problem. UMIACS-TR-99-31, University of Maryland.

Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S., & Bianchini, R. (2024). Splitwise: Efficient generative LLM inference using phase splitting. ISCA.

Qin, R., et al. (2025). Mooncake: Trading more storage for less computation: A KVCache-centric architecture for serving LLM chatbot. FAST.

Robson, J. M. (1981). Worst case analysis of memory allocation strategies. Proceedings of the 2nd Annual ACM Symposium on Theory of Computing, 122-130.

Srivatsa, V., He, Z., Abhyankar, R., Li, D., & Zhang, Y. (2024). Preble: Efficient distributed prompt scheduling for LLM serving. arXiv:2407.00023.

Sun, B., Huang, Z., Zhao, H., Xiao, W., Zhang, X., Li, Y., & Lin, W. (2024). Llumnix: Dynamic scheduling for large language model serving. OSDI.

Agrawal, A., et al. (2024). Vidur: A large-scale simulation framework for LLM inference. MLSys.

Chen, L., Ye, Z., Wu, Y., Zhuo, D., Ceze, L., & Krishnamurthy, A. (2024). Punica: Multi-tenant LoRA serving. MLSys.

Zheng, L., Yin, L., Xie, Z., et al. (2024). SGLang: Efficient execution of structured language model programs. NeurIPS.

Wu, B., Zhong, Y., Zhang, Z., Liu, S., Liu, F., Sun, Y., Huang, G., Liu, X., & Jin, X. (2026). FastServe: Iteration-level preemptive scheduling for large language model inference. NSDI.

Smith, W. E. (1956). Various optimizers for single-stage production. Naval Research Logistics Quarterly, 3(1-2), 59-66.

Software and reproducibility

Code, generated workloads, per-instance measurements, and summary tables are included in the repository. From its root run `python experiments/run_synthetic_experiments.py`, `python experiments/run_exact_small.py`, and `python experiments/run_ilp_scaling.py`. The team's original sample-schedule evaluator remains available as `python experiments/run_experiments.py`. The tested environment was Python 3.13.5 on Windows; the MILP uses SciPy's HiGHS interface (`python -m pip install scipy`), and manuscript generation uses ReportLab. Rebuild the PDF with `python paper/build_manuscript.py`. The paper reads the checked-in experiment CSV files. The references cover LLM serving, scheduling theory, and dynamic storage allocation.